

Prezentacja

Poprawa znalezionych uchybień, Nawiązywanie połączenia, przesyłanie konfiguracji przez sieć



Naprawienie dwóch uchybów znalezionych przy code-review

```
RegulatorPID::RegulatorPID(double k)
: kP_(k)
, Ti_(0.0)
, Td_(0.0)
, uP_(0.0)
, uI_(0.0)
, uD_(0.0)
, ePrev_(0.0)
, sumE_(0.0)
, trybCalk_(LiczCalk::Zewnetrzne)
{
}

RegulatorPID::RegulatorPID(double k, double Ti)
: kP_(k)
, Ti_(Ti)
, Td_(0.0)
, uP_(0.0)
, uI_(0.0)
, uD_(0.0)
, ePrev_(0.0)
, sumE_(0.0)
, trybCalk_(LiczCalk::Zewnetrzne)
{
}

RegulatorPID::RegulatorPID(double k, double Ti, double Td)
: kP_(k)
, Ti_(Ti)
, Td_(Td)
, uP_(0.0)
, uI_(0.0)
, uD_(0.0)
, ePrev_(0.0)
, sumE_(0.0)
, trybCalk_(LiczCalk::Zewnetrzne)
{
}
```



```
RegulatorPID::RegulatorPID(double k, double Ti = 0.0, double Td = 0.0)
: kP_(k)
, Ti_(Ti)
, Td_(Td)
, uP_(0.0)
, uI_(0.0)
, uD_(0.0)
, ePrev_(0.0)
, sumE_(0.0)
, trybCalk_(LiczCalk::Zewnetrzne)
{
}
```

```
// Pomocnicze
double ModelARX::clamp(double v, double vmin, double vmax)
{
    if (v < vmin) return vmin;
    if (v > vmax) return vmax;
    return v;
}
```



```
const double uWe = ograniczU_ ? std::clamp(u, uMin_, uMax_) : u;
```

```
#include <cmath>

namespace
{
    constexpr double PI = 3.14159265358979323846;
}
```

| Usunięcie def.
| I użycie
| <cmath>
| M_PI
V

```
case Typ::Sinus:
{
    const double arg = static_cast<double>(idx) * 2.0 * M_PI / static_cast<double>(N);
    const double w = A_ * std::sin(arg) + S_;
    return w;
}
```

Nawiązywanie połączenia:

Konfiguracja połączenia sieciowego

Konfiguracja Instancji

Typ instancji: Regulator (taktuje)

Tryb sieciowy: Serwer (nasłuchuje)

Parametry połączenia - serwer

Połączenie sieciowe

Port TCP: 5555

Ostrzeżenie: Przejście w tryb sieciowy zablokuje część lokalnych kontrolerek zgodnie ze specyfikacją!

Rozpocznij Połączenie Anuluj

Konfiguracja połączenia sieciowego

Konfiguracja Instancji

Typ instancji: Obiekt (tylko odpowiada)

Tryb sieciowy: Klient (łączy do serwera)

Parametry połączenia - klient

Połączenie sieciowe

Adres IP: 10.0.0.220

Port TCP: 5555

Wykryte serwery

10.0.0.220:5555

Ostrzeżenie: Przejście w tryb sieciowy zablokuje część lokalnych kontrolerek zgodnie ze specyfikacją!

Rozpocznij Połączenie Anuluj

Tryb sieciowy: Regulator
Stan połączenia: Klient połączony:
10.0.0.220:60371

Telemetria:
Wysłano: 10.22 kb
Odebrano: 0 b

● Symulacja wyrabia

Tryb sieciowy: Obiekt
Stan połączenia: Klient: 10.0.0.220:5555

Telemetria:
Wysłano: 0 b
Odebrano: 20.45 kb

● Symulacja wyrabia



Nawiązywanie połączenia:

```
bool PolaczenieSieciowe::polaczZSerwerem(const QString& ip, uint16_t port, int timeoutMs)
{
    rozlacz(false);

    m_aktywnyPortSerwera = 0;
    m_timerBroadcast->stop();
    m_buforOdbioru.clear();

    m_gniazdo = new QTcpSocket(this);
    skonfigurujGniazdoTcp(m_gniazdo);

    m_gniazdo->connectToHost(ip, port);
    emit stanZmieniony("Łączenie z " + ip + ":" + QString::number(port));

    if (!m_gniazdo->waitForConnected(timeoutMs)) {
        const QString powod = m_gniazdo
                                ? m_gniazdo->errorString()
                                : QString("Przerwano próbę połączenia.");
        emit stanZmieniony("Nie udało się połączyć z serwerem.");
        emit zdarzenieTCP("Błąd łączenia: " + powod);

        if (m_gniazdo) {
            disconnect(m_gniazdo, nullptr, this, nullptr);
            m_gniazdo->abort();
            m_gniazdo->deleteLater();
            m_gniazdo = nullptr;
        }

        return false;
    }
}
```

```
bool PolaczenieSieciowe::uruchomSerwer(uint16_t port)
{
    rozlacz(false);

    m_aktywnyPortSerwera = port;

    if (m_serwer->listen(QHostAddress::AnyIPv4, port)) {
        emit stanZmieniony("Serwer nasłuchuje na porcie " + QString::number(port));
        emit zdarzenieTCP("Serwer uruchomiony.");
    }
}
```



[INSTANCJA 1 (Klient)]

[INSTANCJA 2 (Serwer)]

```
|
| === 1. WYKRYWANIE (QUdpSocket) =====
|
| <----- UDP BROADCAST -----
|         "SERVER_UAR:<port>" (co 1s)
|         "WPISANE_IP:<port>" (ręcznie)
|
| === 2. ŁĄCZENIE (QTcpSocket / QTcpServer) =====
|
| ----- TCP CONNECT ----->
| m_gniazdo->connectToHost(IP, port)
| <----- ZAAKCEPTOWANO -----
|
|
| === 3. WYMIANA RAMEK (Protokół UAR) =====
|
| ===== RAMKA: KONFIGURACJA =====>
| Nagłówek : [ Typ: Konfiguracja | Rozmiar: N bajtów]
| Payload   : [ JSON: np. {"kP":7.77, "TTms":123} ]
|
| <===== RAMKA: PRÓBKA SYMULACJI =====
| Nagłówek : [ Typ: ProbkaSymulacji | Rozmiar: stały]
| Payload   : [ struct: rola, krok, t, w, y ]
|
| ===== RAMKA: STEROWANIE =====>
| Nagłówek : [ Typ: Sterowanie | Rozmiar: stały ]
| Payload   : [ struct: rola, krok, u, uP, uI, uD ]
|
|
| === 4. ROZŁĄCZANIE (QTcpSocket) =====
|
| ----- TCP DISCONNECT ----->
| m_gniazdo->abort() lub błąd sieci
| m_buforOdbioru.clear()
```

m_serwer->nextPendingConnection()

onRozlaczono() / onBladGniazda()

m_buforOdbioru.clear()

Diagram sekwencji

Przekazywanie konfiguracji (kod)

```
200 void PolaczenieSieciowe::wyslijRamke(const QByteArray& ramka)
201 {
202     if (m_gniazdo && m_gniazdo->state() == QAbstractSocket::ConnectedState) {
203         qint64 zapisane = m_gniazdo->write(ramka);
204         if (zapisane > 0) {
205             m_wyslaneBajty += static_cast<qint64>(zapisane);
206             emit statystykiZaktualizowane(m_wyslaneBajty, m_odebraneBajty);
207
208             if (zapisane != ramka.size()) {
209                 emit zdarzenieTCP("Uwaga: do bufora TCP trafiła tylko część ramki.");
210             }
211         } else {
212             emit zdarzenieTCP("Błąd wysyłania ramki!");
213         }
214     } else {
215         emit zdarzenieTCP("Brak aktywnego połączenia TCP do wysłania danych!");
216     }
217 }
```

```
16 std::vector<uint8_t> ProtokolUAR::zbudujRamkeWewnetrzna(TypWiadomosci typ, const std::string &payload)
17 {
18     Naglowek naglowek;
19     naglowek.typ = typ;
20     naglowek.rozmiarDanych = static_cast<uint32_t>(payload.size());
21
22     std::vector<uint8_t> ramka;
23     ramka.reserve(sizeof(Naglowek) + payload.size());
24
25     const uint8_t *ptr = reinterpret_cast<const uint8_t*>(&naglowek);
26     ramka.insert(ramka.end(), ptr, ptr + sizeof(Naglowek));
27     ramka.insert(ramka.end(), payload.begin(), payload.end());
28
29     return ramka;
30 }
```

```
98 std::vector<uint8_t> ProtokolUAR::zbudujRamkeKonfiguracji(const KonfiguracjaUAR &cfg)
99 {
100     return zbudujRamkeWewnetrzna(TypWiadomosci::Konfiguracja, cfg.serializujDoJSON());
101 }
```



Protokół

RAMKA SIECIOWA PROTOKOŁU UAR		
NAGŁÓWEK (5 bajtów)	PAYLOAD (Dane)	
TypWiadomosci	rozmiarDanych	
(1 bajt)	(4 bajty)	Rozmiar zgodny z polem "rozmiarDanych"
[uint8_t]	[uint32_t]	Zalezny od typu wiadomości



Protokół

1. Konfiguracja (Typ = 1) -> przesyła string/JSON

```
+-----+
| Zmienna długość zależna od długości tekstu JSON |
| np. { "sym": {"TTms":200}, "pid": {"kp": 1.5, ... }, ... } |
+-----+
```

2. Sterowanie (Typ = 2) -> wymiana u, w, parametrów PID (49 bajtów)

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| Nadawca | Krok   | u   | w   | uP  | uI  | uD  | Flagi |
| (1 bajt) | (8 bajtów)| (8 bajt)| (8 bajt)| (8 bajt)| (8 bajt)| (8 bajt)| (1 bajt) |
+-----+-----+-----+-----+-----+-----+-----+-----+
* Przycisk Reset I wysyła flagę 0x01 (SterowanieResetI)
* Przycisk Reset D wysyła flagę 0x02 (SterowanieResetD)
```

3. ProbkaSymulacji (Typ = 3) -> logowanie stanu (25 bajtów)

```
+-----+-----+-----+-----+-----+
| Nadawca | Krok   | t   | w   | y   |
| (1 bajt) | (8 bajtów)| (8 bajt)| (8 bajt)| (8 bajt)|
+-----+-----+-----+-----+-----+
```